

Module 10 — Stretch: Multi-Service Coordinator (Honors Track)

Honors Track. This is an Honors Track stretch assignment — not required for program completion, but required for Honors distinction. Stretch is for learners who have completed all core assignments, are On Track or Advanced, and are attending consistently.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Prerequisite: Module 10 Applied Lab and Integration Task. This stretch assumes you have already shipped the four-service Compose stack.

Objective

Deploy multiple independent services behind a **coordinator**. The services are:

1. The Module 10 Lab's three endpoints — NLP (Natural Language Processing), KG (Knowledge Graph), and RAG (Retrieval-Augmented Generation) — **repackaged as three separate microservices**, one container per endpoint, each with its own Dockerfile and `lifespan`.
2. A **query-classifier service** that decides which microservice(s) should answer a given question.
3. A **coordinator service** that routes, fans out across the chosen microservices, aggregates the responses, and handles partial failure (one upstream returns 503 → coordinator returns 200 with `"partial": true`).

This stretch is the scaffold for the multi-agent capstone option. The classifier is the equivalent of an agent's "tool selection" step; the coordinator is the orchestration layer that routes between tools and handles their failure modes.

Setup

Use the standalone template repo:

Stretch Thu template repo: [LevelUp-Applied-AI/ml0-s210](#). Click "Use this template" → "Create a new repository" on the template page, name your copy `ml0-s210`, leave Visibility as Public, and create the repository under your own GitHub account.

If you have not already installed the Module 10 packages, go back to the [Setup: install before the concept session](#) section of the reading. This stretch additionally uses `httpx.AsyncClient` (already in the Module 10 `requirements.txt`).

```
git clone https://github.com/<your-username>/ml0-s210.git
cd ml0-s210
git checkout -b stretch-10-thu-coordinator
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

***Hardware sidebar.** This stretch's 5-service stack (three RAG-service replicas + classifier + coordinator) needs ≥16 GB RAM if the generator is loaded once and shared via a model server, or ≥24 GB for naive per-replica model loading. If you are at 16 GB, run the three backends as one shared-model container with multiple workers rather than three independent containers; the stretch spec documents this option.*

Learning outcomes

By completing this stretch you will be able to:

- Decompose a monolithic backend into per-endpoint microservices and document why each split pays.
- Compose a 5-service Docker Compose stack (3 backends + classifier + coordinator) with named networks and explicit `depends_on` chains.
- Implement service-to-service `httpx.AsyncClient` calls with per-call timeouts and structured error mapping.
- Implement the coordinator pattern: classify → fan-out → aggregate → respond.
- Handle partial failure — one upstream returns 503 and the coordinator returns 200 with `"partial": true` and a list of which upstreams responded.
- Frame this as the multi-agent capstone scaffold (the classifier is the tool-selection step; the coordinator is the orchestration layer).

Tasks

1. Decompose the Lab backend

Split the Lab's `api/` into three directories under `services/` — `services/nlp_svc/`, `services/kg_svc/`, `services/rag_svc/` (the starter already nests them this way; `coordinator/` and the data tiers sit at the repo root) — each its own FastAPI app with its own `Dockerfile`, `requirements.txt`, `lifespan`, and single path operation (`/extract`, `/kg/query`, `/rag/answer` respectively). The shared concerns (Pydantic models, `/healthz`, structured errors) are duplicated per service — that is part of the lesson on microservice trade-offs.

2. Author the classifier service

`services/classifier_svc/main.py` exposes `POST /classify` with input `{"question": str}` and output `{"routes": [{"service": "nlp_svc" | "kg_svc" | "rag_svc", "confidence": float}, ...]}`. The starter ships a rules-based classifier (token-overlap heuristic on question shape) as a baseline that returns one entry per matched route with its confidence score; you may replace it with a zero-shot classifier (`facebook/bart-large-mnli`) for the Tier 1 challenge inside this stretch. The coordinator reads `clf_response.json()["routes"]` and selects services by name — keep the shape stable if you replace the classifier.

3. Author the coordinator service

`coordinator/main.py` exposes `POST /answer` with input `{"question": str}` and output `{"results": {service_name: response, ...}, "partial": bool, "responded": {service_name, ...}}`.

The coordinator:

1. Calls `POST classifier:8001/classify` to choose which services to invoke.
2. Fans out concurrent `httpx.AsyncClient` calls to each chosen service with a per-call timeout (default 5 s; RAG gets 10 s because generation dominates).
3. Aggregates the responses. If one upstream returns 503 or times out, the coordinator returns the other responses with `"partial": true` and lists the responding services.
4. If all upstreams fail, returns 503 with structured detail.

4. Compose the 5-service stack

`docker-compose.yml` declares the **5 microservices**: `nlp_svc`, `kg_svc`, `rag_svc`, `classifier_svc`, `coordinator`. The persistence tier (Neo4j + Weaviate) is **optional** for the stretch — wire it in if your `kg_svc` and `rag_svc` need real backends; skip it if your microservices stub their persistence calls. The **graded deliverable is the 5-service coordinator stack**; the optional persistence tier pushes the topology to 7 services total. Healthchecks on every backend; the coordinator depends on the classifier healthy AND the three backend microservices healthy (use `condition: service_healthy` on each — bare `depends_on` lists only wait for container start, not ready).

5. Runbook + observability hooks

Document the end-to-end demo in `README.md`. Add structured request logging on the coordinator with one log line per inbound request including which upstreams were called, which responded, and the total latency. (Module 11 will read this format.)

Tests

CI runs unit-style tests against `httpx` mocks (`tests/test_coordinator.py`) — fast, hermetic, no Docker. The tests cover:

- The `AnswerResponse` and `UpstreamResult` shapes carry the per-source attribution fields the coordinator needs to make routing observable.
- The coordinator enforces a per-call timeout — an upstream call that hangs longer than `timeout_s` returns `status='timeout'` rather than blocking or raising.
- The classifier returns routes with confidence values.

End-to-end routing correctness (KG-shaped → `kg_svc` only; RAG-shaped → `rag_svc`; hybrid → both; partial-failure with `rag_svc` killed mid-run; all-failure → 503) is exercised **manually** by running the full Compose stack locally per the runbook in this guide — the screenshots and runbook output you include in your submission PR are how that part of the work is verified.

Submission

1. Commit your changes to `stretch-10-thu-coordinator` and push.
2. Open a PR within your fork against `main`.
3. Confirm the CI workflow succeeds.

Your PR description must include:

- A one-paragraph summary of the coordinator design.
- A short note on the cost / benefit of the microservice split for this domain (one paragraph — what did splitting the backend buy you, and what did it cost?).
- Confirmation that `docker compose up -d` brings the 5-service (plus data tiers) stack up healthy.
- Output of the partial-failure test (killing `rag_svc` and observing the coordinator's graceful response).
- Paste your PR URL into TalentLMS → Module 10 → Stretch Thu submission to submit this assignment.

Optional enrichment (not graded)

The coordinator pattern naturally extends to an **iterative agent loop**: classify → query → if confidence is low, re-route to a different service or refine the question and retry. This is the next step toward an agent. The stretch README contains a pointer to the capstone bridge — if you wire an iterative loop into the coordinator, you have built the orchestration layer of a multi-tool agent.

